

Mathematics – Linear Algebra for AI (Week 4)

Overview

This chapter focuses on understanding the mathematical foundations behind Linear Regression and Gradient Descent.

It builds on vector and matrix operations to explain how AI models learn from data.

Learning Goal

The objective is to understand how an AI model learns using:

- Matrix operations
- Predictions
- Error computation
- Cost function
- Gradient descent

You must be able to:

- Perform matrix-vector multiplication
 - Understand prediction using linear models
 - Compute error and loss
 - Understand gradients mathematically
 - Implement gradient descent
 - Handle shape issues in NumPy
 - Debug training instability (NaN, divergence)
-

Core Concepts

◆ 1. Matrix × Vector Multiplication (VERY IMPORTANT)

➤ Rule

Multiply each row of the matrix by the vector.

➤ Example

$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

$x = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$

Result:

$$[1 \times 5 + 2 \times 6] = 17$$

$$[3 \times 5 + 4 \times 6] = 39$$

◆ Key Insight

⇒ Matrix $\times$ vector = dot product per row

◆ 2. Shapes in Machine Learning

► Important Shapes

$X \rightarrow (n, \text{features})$

$w \rightarrow (\text{features}, 1)$

$y \rightarrow (n, 1)$

► Rule

(number of columns of X) = (number of rows of w)

◆ 3. Prediction Model

► Formula

$y_{\text{pred}} = Xw + b$

► Interpretation

- X = data
 - w = weights
 - b = bias
 - y_{pred} = prediction
-

◆ 4. Error Vector

► Formula

$\text{error} = y_{\text{pred}} - y$

► Important

⇒ Error is a vector

Each data point has its own error.

◆ 5. Loss Function (MSE)

► Formula

$\text{loss} = \text{mean}(\text{error}^2)$

► Interpretation

- Converts vector $\rightarrow$ scalar
 - Measures global performance
-

◆ Key Insight

error $\rightarrow$ vector

loss $\rightarrow$ scalar

◆ 6. Why Use Transpose?

► Expression

$\text{error}^T \text{error}$

► Purpose

- Converts vector $\rightarrow$ scalar
 - Computes sum of squared errors
-

◆ 7. Gradient (VERY IMPORTANT)

► Formulas

$\text{dw} = (1/n) X^T (y_{\text{pred}} - y)$

$\text{db} = \text{mean}(y_{\text{pred}} - y)$

► Interpretation

- Gradient shows direction of error increase
 - We move in opposite direction
-

◆ 8. Gradient Descent

► Update Rule

$w = w - \text{learning_rate} \times \text{dw}$

$b = b - \text{learning_rate} \times \text{db}$

► Learning Process

1. Predict
 2. Compute error
 3. Compute gradient
 4. Update parameters
-

◆ 9. Learning Rate

► Role

Controls step size during learning

► Effects

Small → slow but stable

Large → fast but unstable (NaN possible)

◆ 10. Why NaN Happens

► Causes

- Learning rate too high
 - Values become too large
 - Overflow → NaN
-

► Solution

- Reduce learning rate
 - Normalize data
-

◆ 11. Feature Normalization

► Formula

$$X = (X - \text{mean}) / \text{std}$$

► Purpose

- Stabilize training
 - Prevent large gradients
-

◆ 12. Reshape (NumPy)

► Example

```
y = y.reshape(-1, 1)
```

► Why?

- Ensure correct matrix operations
 - Avoid shape errors
-

◆ 13. Vector vs Scalar

Vector → multiple values

Scalar → single value

► Example

error → vector

loss → scalar

◆ 14. Training Loop

► Structure

for i in range(1000):

```
y_pred = Xw + b
error = y_pred - y
loss = mean(error2)
```

```
dw = XT error / n
db = mean(error)
```

```
update w, b
```

🧩 Exercises

● Exercise 1: Matrix Multiplication

Given:

A = [[2, 3], [4, 5]]

x = [[1], [2]]

🔗 Compute Ax manually

● Exercise 2: Prediction

Given:

$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

$w = \begin{bmatrix} 0.5 & 1.0 \end{bmatrix}$

$b = 1$

⇒ Compute y_{pred}

Exercise 3: Error and Loss

Given:

$y_{\text{pred}} = [10, 12, 8]$

$y = [9, 11, 7]$

⇒ Compute:

- error
 - squared error
 - loss (MSE)
-

Exercise 4: Gradient

Given:

$X = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$

$\text{error} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$

⇒ Compute:

- X^T
 - $dw = X^T \times \text{error}$
-

Exercise 5: Debugging

Explain why this produces NaN:

$\text{learning_rate} = 1$

⇒ How to fix it?

Exercise 6: Shapes

Given:

$X.\text{shape} = (100, 2)$

👉 What should be the shape of:

- w ?
 - y ?
 - y_{pred} ?
-

🚀 Key Takeaways

- Matrix operations are the foundation of AI
 - Error is a vector, loss is a scalar
 - Gradient guides learning
 - Learning rate controls stability
 - Normalization prevents divergence
 - Shape consistency is critical
-

🧠 Final Insight

Machine Learning = Linear Algebra + Optimization

👉 Data → Matrix

👉 Model → Equation

👉 Learning → Gradient Descent

👤💻 Author

Yann-Méric SOSSOU

Computer Science Graduate | Future AI Engineer